

Q. Trace the output for the following commands

1) `git add A1.txt`

-> A1.txt file get added into staging area.

2) `ls`

-> list out all files and directory from current location

3) `pwd`

-> display current working directory path

4) `cd A1`

-> navigate from current working directory to A1

5) `mkdir ABC`

-> ABC named directory get created

6) `cd ..`

-> move one step back from current working directory

7) `cd ../../`

-> move two step back from current working directory

8) `git add F1.txt F2.txt`

-> F1.txt and F2.txt get added into staging area

9) `git add --A`

-> all files get added into staging (similar to `git add --a`)

10) `git add -A`

-> This command will give an error

11) `rm Abc.txt`

-> Abc.txt file get deleted from system.

12) `rm ABC` (Where `Abc` is directory)

-> This command will give an error because `ABC` is a directory
if we want to remove a directory then we need to use `-r` option

13) `git rm --cached M1.txt`

-> `M1.txt` get removed from staging area and place back into working area.

14) `git rm -f M1.txt`

-> `M1.txt` get removed from staging area and deleted from system.

15) `git restore Sunday.txt`

-> `Sunday.txt` file get restored to previous state.

16) `git restore --staged Sunday.txt`

-> `Sunday.txt` get removed from staging area and placed in working area.

Q. State the following statements are valid or invalid

1) In linux file type is depends on its extention

-> invalid

-> valid statement: In Linux file type is depends on its content

2) Every linux command start with git?

-> invalid

-> valid statement: Linux does not start with git

3) Linux File system and windows file system are same.

-> invalid

-> valid statement: Linux File system and windows file system are different

4) git is maintained by windows

-> invalid

-> Valid statement: git is maintained by windows

5) github is maintained by Linux

-> invalid

-> Valid statement: From 2018 github is maintained by windows

6) we can execute git command on normal directory

-> invalid

-> Valid statement: git commands are executed on git repository.

7) Git bash is version control tool.

-> invalid

-> Valid statement: git bash is software used to interact with git

8) git and gitbash are same.

-> invalid

-> Valid statement: git and gitbash are different

9) local repository is also called as commit area.

-> valid

10) working area files(untracked,modified) we push into staging area by using "git commit" command

-> invalid

-> valid statement: we add untracked or modified files into staging area by using git add command

11) 'git restore --staged' command remove a file from commit area and put into staging area.

-> invalid

-> valid statement: git restore --staged command remove a file from staging area and put into working area.

12) We can add Untracked files directly into commit area.

-> invalid, First add into staging area, then add into commit area.

13) We can add modified files directly into commit area.

-> invalid, First add into staging area, then add into commit area.

14) If local repository file is deleted from our system then no need to add in commit area.

-> invalid

-> Valid statement: If local repository file is deleted from our system then we need to add in commit area.

15) git bash display committed file name is blue color.

-> invalid (committed files not shown in the status history)

16) git bash display modified file name is red color

-> valid

17) git bash display Untracked Files in green color

-> invalid

18) when we add a file into staging area then index is created or index is updated

-> valid

19) To check the index of file we use command "git ls-files -s File_Name"

-> Valid

20) Every file have unique index.

-> invalid

-> Valid statement: Index of file created based on content. If two file have same content then can have same index

Git Branch

Q. What is mean by Branch?

-> An independent line of development is considered as branch.

Q. Which branch is considered as Default Branch in git?

-> When we initialize a Git repository, Git creates a default branch, usually named master or main.

*) Git HEAD and HEAD file

Q. What is mean by HEAD in git?

-> HEAD is pointer and it points to current branch on which you are working.

When you create new branch or switch to new branch then HEAD is automatically point to that branch.

Q. What is The HEAD File?

-> The HEAD file is a simple text file and Keep Track of our Current Branch.

ref: refs/heads/master

-> HEAD file is present in .git folder

-> HEAD file get updated automatically whenever you switch branches or checkout a specific commit.

commands related to branch

1) git branch

It will List out all the branches from current repository

Syntax: git branch

2) Creating a new branch:

If we want to create a branch in local repository then use "git branch" command in below format.

Syntax: git branch brach_name

Example: git branch feature-123

Here, feature-123 named branch get created. If If specified brach_name is already available in the local respository then we will get an error saying " a branch named 'xxx' already exists

```
Dell@DESKTOP-1VT9LL4 MINGW64 /e/DevOps/git-practice (master)
$ git branch feature-001
fatal: a branch named 'feature-001' already exists
```

Note: Branch name is not case sensitive.
i.e feature-123 is same as FEATURE-123

Switching from one branch to another branch

-> To switch from one branch to another branch we have two commands

- 1) git switch
- 2) git checkout

1) git switch command:

It is used to switch from current branch to specified branch.

Syntax: git switch branch_name

Example: git switch feature-001

Practical Demo:

```
$ git switch feature-001
hint: core.useBuiltinFSMonitor=true is deprecated; please use 'git config --global core.fsmonitor' instead
Switched to branch 'feature-001'
A       F1.txt
M       F2.txt
A       a1
A       a2
A       a3
```

* `git switch -c branch_name`

This command creates a specified branch and switch to that branch.
here -c stands for create

Example: `git switch -c feature-002`

Practical Demo

```
De11@DESKTOP-1VT9LL4 MINGW64 /e/DevOps/git-practice (feature-001)
$ git switch -c feature-002
switched to a new branch 'feature-002'

De11@DESKTOP-1VT9LL4 MINGW64 /e/DevOps/git-practice (feature-002)
$ |
```

* `git checkout brach_name`

This command is used to switch from current branch to specified branch.
Checkout a branch means that you are going to work on that branch

Example:
`git checkout feature-001`

* `git checkout -b brach_name`

This command create new branch and switch to newly created branch.

Example:
`git checkout -b feature-003`

Note: Commit your changes before switching to a branch

* **Delete a branch:** We can delete a branch by using below command

`git branch -d branch-name`

*) **Rename a branch:** To rename a branch use below command

`git branch -m old_name new_name`

Example: `git branch feature-001 feature-new-001`